ಪರ್ಸೆಪ್ಟ್ರಾನ್‌ನಿಂದ ಬ್ಯಾಕ್‌ಪ್ರೊಪಗೇಶನ್‌ವರೆಗೆ

ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಹೇಗೆ ಕಲಿಯುತ್ತದೆ?

ಸರಳ ಕನ್ನಡದಲ್ಲಿ, ದಿನನಿತ್ಯದ ಉದಾಹರಣೆಗಳು ಮತ್ತು ಹಂತ-ಹಂತದ ಲೆಕ್ಕಗಳೊಂದಿಗೆ

“Lecture 3: Foundations of Learning in Neural Networks — From Perceptron to Backpropagation”

ಉಪನ್ಯಾಸ ಸ್ಲೈಡ್‌ಗಳನ್ನು ಆಧರಿಸಿದ ಕನ್ನಡ ಅಧ್ಯಯನ ಮಾರ್ಗದರ್ಶಿ

ಪರಿವಿಡಿ

೧. ದೊಡ್ಡ ಚಿತ್ರ: ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವ ಚಕ್ರ

೨. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಎಂದರೇನು?

೩. ಉದಾಹರಣೆ: ವಿದ್ಯಾರ್ಥಿಯ ಕಾರು ಖರೀದಿ ನಿರ್ಧಾರ

೪. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಒಂದು ರೇಖೆ ಎಳೆಯುತ್ತದೆ (ರೇಖೀಯ ವರ್ಗೀಕರಣ ಮತ್ತು ಬಯಾಸ್)

೫. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ತಪ್ಪಿನಿಂದ ಕಲಿಯುತ್ತದೆ (ರೋಸೆನ್‌ಬ್ಲಾಟ್ ನಿಯಮ)

೬. XOR ಸಮಸ್ಯೆ — ಒಂದು ರೇಖೆ ಸಾಲದಿದ್ದಾಗ

೭. Activation function ಮತ್ತು ಮುಂದಕ್ಕೆ ಸಂಕೇತ ಹರಿವು (Feedforward)

೮. ಮೂರು ಪದರದ ನೆಟ್‌ವರ್ಕ್: ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನಿಂದ ಲೆಕ್ಕ

೯. ತಪ್ಪನ್ನು (Error) ಅಳೆಯುವುದು: ಶಿಕ್ಷಕರೊಂದಿಗೆ ಕಲಿಕೆ

೧೦. ತಪ್ಪನ್ನು ಹಿಂದಕ್ಕೆ ಹಂಚುವುದು (Backpropagation ನ ಮೂಲ ಕಲ್ಪನೆ)

೧೧. ತೂಕ ಬದಲಿಸುವ ಸರಿಯಾದ ದಾರಿ: Gradient Descent

೧೨. Learning rate, ಸ್ಥಳೀಯ ಕನಿಷ್ಠ, Vanishing / Exploding gradients

೧೩. Backpropagation ಸೂತ್ರದ ಉತ್ಪತ್ತಿ ಮತ್ತು ತೂಕದ ನಿಜವಾದ ಬದಲಾವಣೆ

೧೪. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

೧. ದೊಡ್ಡ ಚಿತ್ರ: ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವ ಚಕ್ರ

ಈ ಉಪನ್ಯಾಸದ ಎಲ್ಲ ಪಾಠಗಳು ಒಂದೇ ಪ್ರಶ್ನೆಗೆ ಉತ್ತರ ಹುಡುಕುತ್ತವೆ: **ಕಂಪ್ಯೂಟರ್ ಯಾರೂ ಹೇಳಿಕೊಡದೆ, ಉದಾಹರಣೆಗಳನ್ನು ನೋಡಿ ತಾನೇ ಹೇಗೆ ಕಲಿಯುತ್ತದೆ?** ಇದಕ್ಕೆ ಉತ್ತರ ಒಂದು ಸರಳ ಚಕ್ರ:

ಹಂತ	ಇಂಗ್ಲಿಷ್ ಹೆಸರು	ಏನಾಗುತ್ತದೆ?
೧	Forward pass	ಇನ್‌ಪುಟ್ ಅನ್ನು ತೂಕಗಳ ಮೂಲಕ ಹಾಯಿಸಿ, ನೆಟ್‌ವರ್ಕ್ ತನ್ನ ಉತ್ತರ (ಊಹೆ) ಕೊಡುತ್ತದೆ
೨	Compare	ಆ ಉತ್ತರವನ್ನು ಸರಿಯಾದ ಉತ್ತರದೊಂದಿಗೆ (label) ಹೋಲಿಸುತ್ತದೆ
೩	Error	ಎಷ್ಟು ತಪ್ಪಾಯಿತು ಎಂಬುದನ್ನು ಒಂದು ಸಂಖ್ಯೆಯಲ್ಲಿ ಅಳೆಯುತ್ತದೆ
೪	Backpropagation	ಆ ತಪ್ಪಿಗೆ ಯಾವ ತೂಕ ಎಷ್ಟು ಕಾರಣ ಎಂದು ಹಿಂದಕ್ಕೆ ಹಂಚುತ್ತದೆ
೫	Weight update	ತಪ್ಪು ಕಡಿಮೆ ಆಗುವಂತೆ ತೂಕಗಳನ್ನು ಸ್ವಲ್ಪ ಬದಲಿಸುತ್ತದೆ
೬	Repeat	ಇದನ್ನು ಸಾವಿರಾರು ಬಾರಿ ಪುನರಾವರ್ತಿಸುತ್ತದೆ

> **ಕಥೆ: ಸೈಕಲ್ ಕಲಿಯುವ ಮಗು**
> ಪುಟ್ಟ ಮಗು ಮೊದಲ ಬಾರಿ ಸೈಕಲ್ ಹತ್ತುತ್ತದೆ. ಪೆಡಲ್ ತುಳಿಯುತ್ತದೆ (*forward*), ಸೈಕಲ್ ಎಡಕ್ಕೆ ವಾಲುತ್ತದೆ. ಎಷ್ಟು ವಾಲಿತು ಎಂದು ಗಮನಿಸುತ್ತದೆ (*error*). "ಹ್ಯಾಂಡಲ್ ಅನ್ನು ಬಲಕ್ಕೆ ಎಷ್ಟು ತಿರುಗಿಸಬೇಕಿತ್ತು? ಕಾಲಿನ ಬಲ ಎಷ್ಟಿರಬೇಕಿತ್ತು?" ಎಂದು ಯೋಚಿಸುತ್ತದೆ (*backprop: ತಪ್ಪಿಗೆ ಯಾರು ಕಾರಣ?*). ಮುಂದಿನ ಬಾರಿ ಸ್ವಲ್ಪ ಮಾತ್ರ ಹ್ಯಾಂಡಲ್ ತಿರುಗಿಸುತ್ತದೆ (*weight update*). ಹತ್ತಾರು ಬಾರಿ ಬಿದ್ದು ಏಳುತ್ತಾ ಕೊನೆಗೆ ಸೈಕಲ್ ಓಡಿಸುತ್ತದೆ (*repeat*). ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್ ಕಲಿಯುವುದೂ ಹೀಗೆಯೇ — ಒಂದೇ ವ್ಯತ್ಯಾಸ: ಮಗುವಿನ ಬದಲು ಸಂಖ್ಯೆಗಳು ಮತ್ತು ಗಣಿತ.

೨. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಎಂದರೇನು?

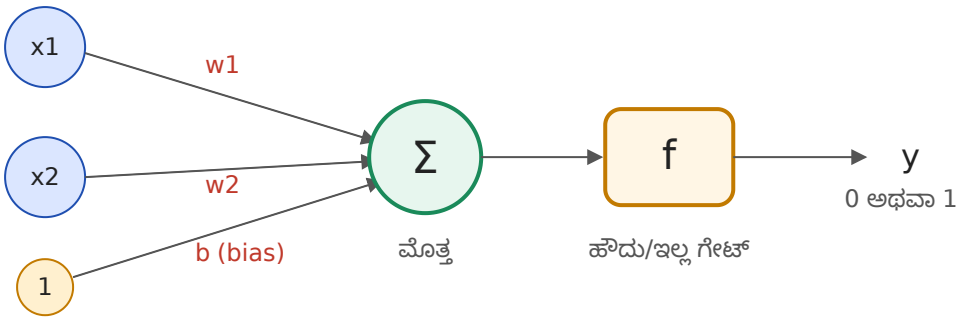
ಇತಿಹಾಸ ಮತ್ತು ಸ್ಫೂರ್ತಿ

೧೯೫೦–೬೦ ರ ದಶಕದಲ್ಲಿ ಫ್ರಾಂಕ್ ರೋಸೆನ್‌ಬ್ಲಾಟ್ (Frank Rosenblatt) ಮೆದುಳಿನ ನರಕೋಶ (neuron) ಹೇಗೆ ಕೆಲಸ ಮಾಡುತ್ತದೆ ಎಂಬುದರಿಂದ ಸ್ಫೂರ್ತಿ ಪಡೆದು **ಪರ್ಸೆಪ್ಟ್ರಾನ್** ಎಂಬ ಮೊದಲ ಕಲಿಯಬಲ್ಲ ಯಂತ್ರವನ್ನು ಮಾಡಿದರು. ಅವರ ಯಂತ್ರದಲ್ಲಿ ಮೂರು ಭಾಗಗಳಿದ್ದವು:

ಭಾಗ	ಕೆಲಸ	ಮನುಷ್ಯನ ಹೋಲಿಕೆ
S (Sensory)	ಹೊರಗಿನ ಸಂಕೇತ ಗ್ರಹಿಸುವ ಘಟಕಗಳು (ಫೋಟೊಸೆಲ್‌ಗಳು)	ಕಣ್ಣಿನ ಪರದೆ, ಕಿವಿ
A (Associative)	ಸಂಕೇತಗಳನ್ನು ಕೂಡಿ, ಒಂದು ಮಿತಿ (threshold θ) ದಾಟಿದರೆ ಮಾತ್ರ ಮುಂದಕ್ಕೆ ಕಳುಹಿಸುತ್ತದೆ	ನರಕೋಶ
R (Response)	ಕೊನೆಯ ತೀರ್ಮಾನ — ದೀಪ ಹೊತ್ತಿಸು, ಸ್ವಿಚ್ ಒತ್ತು	ಕೈ ಎತ್ತುವುದು, ಮಾತು

ಸರಳ ಅರ್ಥ

ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಎಂದರೆ **ಒಂದೇ ಒಂದು ಕೃತಕ ನರಕೋಶ**. ಅದು ಹೀಗೆ ಯೋಚಿಸುತ್ತದೆ: "ಎಲ್ಲ ಸಂಕೇತಗಳನ್ನು ಅವುಗಳ ಮಹತ್ವದಿಂದ (ತೂಕ) ಗುಣಿಸಿ ಕೂಡಿದಾಗ ಸಾಕಷ್ಟು ದೊಡ್ಡ ಮೊತ್ತ ಬಂದರೆ 'ಹೌದು (1)', ಇಲ್ಲದಿದ್ದರೆ 'ಇಲ್ಲ (0)' ಎನ್ನುತ್ತೇನೆ."



ಚಿತ್ರ ೧: ಪರ್ಸೆಪ್ಟ್ರಾನ್ — ಇನ್‌ಪುಟ್‌ಗಳು × ತೂಕ → ಮೊತ್ತ Σ → ಗೇಟ್ f → ಔಟ್‌ಪುಟ್ y

ಮುಖ್ಯ ಸೂತ್ರ

```
y = θ( w1·x1 + w2·x2 + … + wm·xm + b )

θ  = ಹೆವಿಸೈಡ್ ಸ್ಟೆಪ್ ಫಂಕ್ಷನ್: ಒಳಗಿನ ಮೌಲ್ಯ > 0 ಆದರೆ 1, ಇಲ್ಲದಿದ್ದರೆ 0
w  = ತೂಕಗಳು (ಪ್ರತಿ ಇನ್‌ಪುಟ್‌ನ ಮಹತ್ವ),   b = ಬಯಾಸ್,   m = ಇನ್‌ಪುಟ್‌ಗಳ ಸಂಖ್ಯೆ
```

ಹೋಲಿಕೆ: ಕೆ.ಆರ್.ಎಸ್. ಅಣೆಕಟ್ಟು

ಕಾವೇರಿಯ ಉಪನದಿಗಳಿಂದ ಬರುವ ನೀರು ಒಂದು ಮಟ್ಟ ತಲುಪುವವರೆಗೆ ಅಣೆಕಟ್ಟಿನ ಗೇಟ್ ತೆರೆಯುವುದಿಲ್ಲ. ಮಟ್ಟ ದಾಟಿದ ತಕ್ಷಣ ಗೇಟ್ ತೆರೆಯುತ್ತದೆ (ಔಟ್‌ಪುಟ್ = 1). ಪ್ರತಿ ನದಿಯ ನೀರಿಗೆ ಬೇರೆ ಬೇರೆ ಮಹತ್ವ (ತೂಕ) ಇರಬಹುದು. ಪರ್ಸೆಪ್ಟ್ರಾನ್‌ನ ಆಕ್ಟಿವೇಶನ್ ಇದೇ ರೀತಿ ಕೆಲಸ ಮಾಡುತ್ತದೆ.

೩. ಉದಾಹರಣೆ: ವಿದ್ಯಾರ್ಥಿಯ ಕಾರು ಖರೀದಿ ನಿರ್ಧಾರ

ಸನ್ನಿವೇಶ: ಬೆಂಗಳೂರಿನ ಒಬ್ಬ ವಿದ್ಯಾರ್ಥಿಗೆ ಸೆಕೆಂಡ್‌ಹ್ಯಾಂಡ್ ಕಾರು ಕೊಳ್ಳಬೇಕೇ ಬೇಡವೇ ಎಂದು ನಿರ್ಧರಿಸಬೇಕು. ಅವನು ಮೂರು ಪ್ರಶ್ನೆಗಳನ್ನು ಕೇಳಿಕೊಳ್ಳುತ್ತಾನೆ (ಹೌದು = 1, ಇಲ್ಲ = 0):

ಇನ್‌ಪುಟ್	ಪ್ರಶ್ನೆ	ತೂಕ (ಮಹತ್ವ)	ಅರ್ಥ
x1	ಬೆಲೆ ಕಡಿಮೆ ಇದೆಯೇ?	w1 = **0.6**	ಬೆಲೆ ಅತಿ ಮುಖ್ಯ
x2	ಕಾರು 5 ವರ್ಷದೊಳಗಿನದೇ?	w2 = **0.4**	ವಯಸ್ಸು ಸ್ವಲ್ಪ ಮುಖ್ಯ
x3	ಕಾರಿಗೆ ಹಾನಿ (damage) ಆಗಿದೆಯೇ?	w3 = **−0.5**	ಹಾನಿ ಇದ್ದರೆ ನಿರ್ಧಾರ ವಿರುದ್ಧ (ಋಣ ತೂಕ)

ನಿಯಮ: ಮೊತ್ತ Σ = w1·x1 + w2·x2 + w3·x3. **Σ ≥ 0.5 ಆದರೆ ಕೊಳ್ಳಿ (1), ಇಲ್ಲದಿದ್ದರೆ ಬೇಡ (0).** (ಸರಳತೆಗಾಗಿ ಬಯಾಸ್ ಬಿಟ್ಟು ಮಿತಿಯನ್ನು 0.5 ಎಂದು ನೇರವಾಗಿ ಇಟ್ಟಿದ್ದೇವೆ.)

ಹಂತ-ಹಂತದ ಲೆಕ್ಕ

ಪರಿಸ್ಥಿತಿ	(x1, x2, x3)	ಲೆಕ್ಕ	Σ	ತೀರ್ಮಾನ
ಬೆಲೆ ಕಡಿಮೆ, ಹಳೆಯ ಕಾರು, ಹಾನಿ ಇಲ್ಲ	(1, 0, 0)	0.6×1 + 0.4×0 − 0.5×0	0.6	**ಕೊಳ್ಳಿ (1)**
ಬೆಲೆ ಹೆಚ್ಚು, ಹೊಸ ಕಾರು, ಹಾನಿ ಇಲ್ಲ	(0, 1, 0)	0.6×0 + 0.4×1 − 0.5×0	0.4	**ಬೇಡ (0)**
ಬೆಲೆ ಕಡಿಮೆ, ಹಳೆಯ ಕಾರು, ಹಾನಿ ಇದೆ	(1, 0, 1)	0.6×1 + 0.4×0 − 0.5×1	0.1	**ಬೇಡ (0)**
ಬೆಲೆ ಕಡಿಮೆ, ಹೊಸ ಕಾರು, ಹಾನಿ ಇದೆ	(1, 1, 1)	0.6×1 + 0.4×1 − 0.5×1	0.5	**ಕೊಳ್ಳಿ (1) — ಮಿತಿಗೆ ಸಮ**
ಬೆಲೆ ಕಡಿಮೆ, ಹೊಸ ಕಾರು, ಹಾನಿ ಇಲ್ಲ	(1, 1, 0)	0.6×1 + 0.4×1 − 0.5×0	1.0	**ಕೊಳ್ಳಿ (1) — ಅತ್ಯುತ್ತಮ**

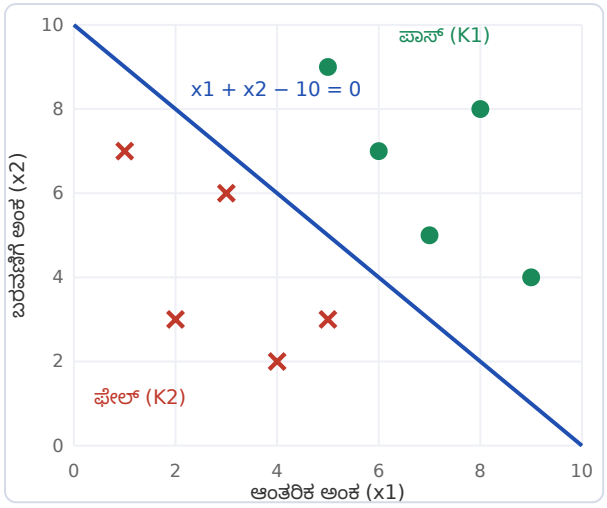
> **ಈ ಉದಾಹರಣೆಯಿಂದ ಕಲಿಯುವುದು**
>
> • **ಧನಾತ್ಮಕ ತೂಕ** “ಹೌದು” ಎನ್ನುವ ಕಡೆ ತಳ್ಳುತ್ತದೆ; **ಋಣ ತೂಕ** “ಬೇಡ” ಎನ್ನುವ ಕಡೆ ತಳ್ಳುತ್ತದೆ.
> • ತೂಕ ದೊಡ್ಡದಾದಷ್ಟೂ ಆ ಅಂಶದ ಪ್ರಭಾವ ಹೆಚ್ಚು.
> • ನ್ಯೂರಲ್ ನೆಟ್‌ವರ್ಕ್‌ನ “ಕಲಿಕೆ” ಎಂದರೆ ಸರಿಯಾದ ತೂಕಗಳನ್ನು (0.6, 0.4, −0.5 ನಂತಹವು) ಕಂಡುಹಿಡಿಯುವುದೇ.

೪. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಒಂದು ರೇಖೆ ಎಳೆಯುತ್ತದೆ (ರೇಖೀಯ ವರ್ಗೀಕರಣ ಮತ್ತು ಬಯಾಸ್)

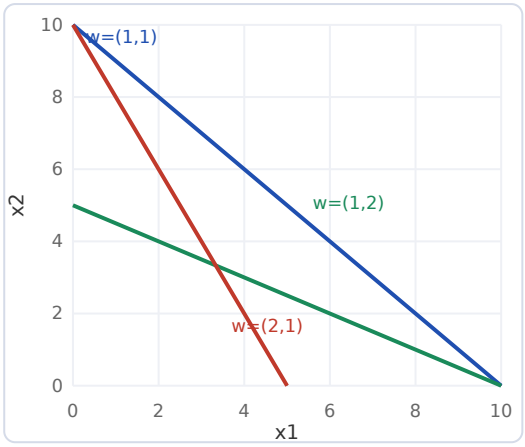
ಪರ್ಸೆಪ್ಟ್ರಾನ್ ವಾಸ್ತವವಾಗಿ ಎರಡು ಗುಂಪುಗಳ ನಡುವೆ ಒಂದು **ನೇರ ರೇಖೆ** ಎಳೆಯುತ್ತದೆ. ರೇಖೆಯ ಒಂದು ಬದಿ ವರ್ಗ K1 (ಉತ್ತರ 1), ಇನ್ನೊಂದು ಬದಿ K2 (ಉತ್ತರ 0). ರೇಖೆಯ ಸಮೀಕರಣ: `w1·x1 + w2·x2 = 0`.

ಪ್ರತ್ಯಕ್ಷ ಉದಾಹರಣೆ: ಪರೀಕ್ಷೆಯಲ್ಲಿ ಪಾಸ್ ಅಥವಾ ಫೇಲ್

ಒಂದು ವಿಷಯದಲ್ಲಿ **ಆಂತರಿಕ ಅಂಕ (x1)** ಮತ್ತು **ಬರವಣಿಗೆ ಅಂಕ (x2)** ಇವೆ (ಪ್ರತಿಯೊಂದೂ 10 ರಲ್ಲಿ). ಎರಡರ ಮೊತ್ತ 10 ಕ್ಕಿಂತ ಹೆಚ್ಚಿದ್ದರೆ ಪಾಸ್. ಆಗ w1 = 1, w2 = 1 ಮತ್ತು ಬಯಾಸ್ b = −10.



ಚಿತ್ರ ೨: ಹಸಿರು ವೃತ್ತ = ಪಾಸ್, ಕೆಂಪು × = ಫೇಲ್, ನೀಲಿ ರೇಖೆ = ನಿರ್ಧಾರ ರೇಖೆ



ಚಿತ್ರ ೩: ತೂಕಗಳ ಅನುಪಾತ ಬದಲಾದರೆ ರೇಖೆ ತಿರುಗುತ್ತದೆ

ವಿದ್ಯಾರ್ಥಿ	(x1, x2)	ಲೆಕ್ಕ 1·x1 + 1·x2 − 10	ಫಲಿತಾಂಶ
ಅನಿತಾ	(6, 7)	6 + 7 − 10 = 3	**3 > 0 → ಪಾಸ್**
ಬಸು	(3, 4)	3 + 4 − 10 = −3	**−3 ≤ 0 → ಫೇಲ್**
ಚೇತನ್	(5, 5)	5 + 5 − 10 = 0	**ರೇಖೆಯ ಮೇಲೆ; ಸ್ಟೆಪ್ ಫಂಕ್ಷನ್‌ನಲ್ಲಿ ಮೌಲ್ಯ > 0 ಅಲ್ಲ → ಫೇಲ್**

ಇಳಿಜಾರು (slope) ಮತ್ತು ಬಯಾಸ್

- ಸಮೀಕರಣವನ್ನು x2 ಗಾಗಿ ಬಿಡಿಸಿದರೆ: `x2 = −(w1/w2)·x1 − b`. ಇಳಿಜಾರು **k = −w1/w2**. ನಮ್ಮ ಉದಾಹರಣೆಯಲ್ಲಿ k = −1/1 = −1.
- **ತೂಕಗಳು** ರೇಖೆಯನ್ನು **ತಿರುಗಿಸುತ್ತವೆ** (ಚಿತ್ರ ೩).
- **ಬಯಾಸ್** ರೇಖೆಯನ್ನು ಇಳಿಜಾರು ಬದಲಿಸದೆ **ಎತ್ತಿ/ಇಳಿಸಿ ಸರಿಸುತ್ತದೆ**. ಬಯಾಸ್ ಇಲ್ಲದಿದ್ದರೆ ರೇಖೆ ಯಾವಾಗಲೂ ಮೂಲಬಿಂದು (0,0) ಮೂಲಕವೇ ಹಾದುಹೋಗಬೇಕಾಗುತ್ತಿತ್ತು. ಅದಕ್ಕೆ ಇನ್‌ಪುಟ್ “1” ಮತ್ತು ತೂಕ w3 ಇರುವ ಹೆಚ್ಚುವರಿ ನೋಡ್ ಸೇರಿಸುತ್ತೇವೆ: `w1x1 + w2x2 + w3·1 = 0`.

> **ಹೋಲಿಕೆ**
>
> ಎರಡು ಊರುಗಳ ನಡುವೆ ಗಡಿ ರೇಖೆ ಹಾಕುವಂತೆ: ತೂಕಗಳು ಗಡಿಯ ದಿಕ್ಕನ್ನು ನಿರ್ಧರಿಸುತ್ತವೆ, ಬಯಾಸ್ ಗಡಿಯನ್ನು ಯಾವ ಕಡೆ ಇಡಬೇಕು ಎಂದು ನಿರ್ಧರಿಸುತ್ತದೆ.

೫. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ತಪ್ಪಿನಿಂದ ಕಲಿಯುತ್ತದೆ (ರೋಸೆನ್‌ಬ್ಲಾಟ್ ನಿಯಮ)

ಇದುವರೆಗೆ ತೂಕಗಳನ್ನು ನಾವೇ ಕೊಟ್ಟೆವು. ಈಗ ಪರ್ಸೆಪ್ಟ್ರಾನ್ **ತಾನೇ** ಕಲಿಯುವ ನಿಯಮ ನೋಡೋಣ. ತತ್ವ ತುಂಬ ಸರಳ:

- ಉತ್ತರ ಸರಿಯಾಗಿದ್ದರೆ → ಏನೂ ಮಾಡಬೇಡ.
- ಉತ್ತರ ತಪ್ಪಾಗಿದ್ದರೆ → ತೂಕಗಳನ್ನು ಬದಲಿಸು.

> **ಕಲಿಕೆ ನಿಯಮ**
>
> ```
> w_ಹೊಸ = w_ಹಳೆಯ + η × (t − o) × x
>
> η (ಈಟಾ) = ಕಲಿಕಾ ದರ (learning rate)      t = ಸರಿಯಾದ ಉತ್ತರ (label)
> o = ನೆಟ್‌ವರ್ಕ್ ಕೊಟ್ಟ ಉತ್ತರ                x = ಆ ತೂಕಕ್ಕೆ ಸಂಬಂಧಿಸಿದ ಇನ್‌ಪುಟ್
> ```
>
> ಔಟ್‌ಪುಟ್ 0 ಅಥವಾ 1 ಆಗಿರುವುದರಿಂದ (t − o) ಕೇವಲ **−1, 0 ಅಥವಾ +1** ಆಗಿರುತ್ತದೆ: ಕಡಿಮೆ ಹೇಳಿದರೆ +1 (ತೂಕ ಹೆಚ್ಚಿಸು), ಹೆಚ್ಚು ಹೇಳಿದರೆ −1 (ತೂಕ ಕಡಿಮೆ ಮಾಡು).

ಉದಾಹರಣೆ: “ಆಟಕ್ಕೆ ಹೊರಗೆ ಹೋಗಬಹುದೇ?”

ಅಮ್ಮನ ನಿಯಮ: **ಹೋಂವರ್ಕ್ ಮುಗಿದಿರಬೇಕು (x1)** ಮತ್ತು **ಮಳೆ ಇರಬಾರದು (x2)** — ಎರಡೂ ಸರಿಯಿದ್ದರೆ ಮಾತ್ರ ಹೋಗಬಹುದು (1). ಇದು AND ನಿಯಮ.

x1	x2	ಸರಿಯಾದ ಉತ್ತರ t
0	0	0
0	1	0
1	0	0
1	1	1

ಆರಂಭ: w1 = 0, w2 = 0, b = 0, η = 0.1. ಈ ನಾಲ್ಕು ಸಾಲುಗಳನ್ನು ಪದೇ ಪದೇ ತೋರಿಸುತ್ತೇವೆ (ಪ್ರತಿ ಸುತ್ತಿಗೆ *epoch* ಎನ್ನುತ್ತಾರೆ). **ತಪ್ಪಾದಾಗ ಮಾತ್ರ** ಬದಲಾವಣೆ:

ಸುತ್ತು	ಇನ್‌ಪುಟ್	t	o (ನೆಟ್‌ವರ್ಕ್)	t − o	ಹೊಸ w1, w2, b
1	(1,1)	1	0	+1	0.1, 0.1, 0.1
2	(0,0)	0	1	−1	0.1, 0.1, 0.0
2	(0,1)	0	1	−1	0.1, 0.0, −0.1
2	(1,1)	1	0	+1	0.2, 0.1, 0.0
3	(0,1)	0	1	−1	0.2, 0.0, −0.1
3	(1,0)	0	1	−1	0.1, 0.0, −0.2
3	(1,1)	1	0	+1	0.2, 0.1, −0.1
4	(1,0)	0	1	−1	0.1, 0.1, −0.2
4	(1,1)	1	0	+1	0.2, 0.2, −0.1
5	(0,1)	0	1	−1	0.2, 0.1, −0.2
6	ಯಾವ ತಪ್ಪೂ ಇಲ್ಲ — ಕಲಿಕೆ ಮುಗಿಯಿತು!				

ಅಂತಿಮ ತೂಕಗಳು: w1 = 0.2, w2 = 0.1, b = −0.2. ಪರೀಕ್ಷೆ: (1,1) → 0.2+0.1−0.2 = 0.1 > 0 → **1** ✓; (0,1) → −0.1 → **0** ✓; (1,0) → 0.0 → **0** ✓; (0,0) → −0.2 → **0** ✓.

Perceptron Convergence Theorem

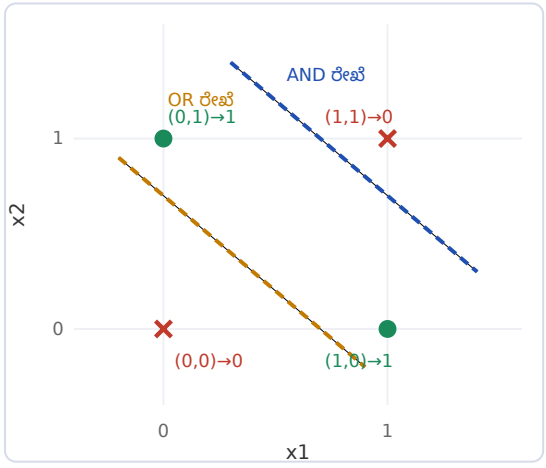
ದತ್ತಾಂಶವನ್ನು ಒಂದು ನೇರ ರೇಖೆಯಿಂದ ಬೇರ್ಪಡಿಸಲು ಸಾಧ್ಯವಿದ್ದರೆ (*linearly separable*) ಮತ್ತು η ಸಾಕಷ್ಟು ಚಿಕ್ಕದಿದ್ದರೆ, ಈ ನಿಯಮ **ಸೀಮಿತ ಸಂಖ್ಯೆಯ ಹೆಜ್ಜೆಗಳಲ್ಲೇ** ಸರಿಯಾದ ತೂಕಗಳನ್ನು ಕಂಡುಹಿಡಿಯುತ್ತದೆ ಎಂದು ಗಣಿತದಿಂದ ಸಾಬೀತಾಗಿದೆ. ಆದರೆ ಈ ನಿಯಮ ಕೇವಲ ಕೊನೆಯ ನ್ಯೂರಾನ್‌ನ ತಪ್ಪನ್ನು ಬಳಸುತ್ತದೆ — ಮೂಲ ಪರ್ಸೆಪ್ಟ್ರಾನ್‌ನಲ್ಲಿ ಮರೆ ಪದರಗಳೇ ಇರಲಿಲ್ಲ. ಅದಕ್ಕೆ ಮಿತಿ ಇದೆ — ಮುಂದಿನ ಭಾಗ ನೋಡಿ.

೬. XOR ಸಮಸ್ಯೆ — ಒಂದು ರೇಖೆ ಸಾಲದಿದ್ದಾಗ

ಕಥೆ: ಇಡ್ಲಿ ಅಥವಾ ದೋಸೆ — ಒಂದೇ!

ಅಮ್ಮ ಹೇಳಿದರು: "ತಿಂಡಿಗೆ ಇಡ್ಲಿ ಅಥವಾ ದೋಸೆ ತಿನ್ನು — **ಒಂದನ್ನು ಮಾತ್ರ**. ಎರಡೂ ಬೇಡ, ಎರಡೂ ಇಲ್ಲದಿರುವುದೂ ಬೇಡ." ಅಂದರೆ ಇಡ್ಲಿ (x1) ಮತ್ತು ದೋಸೆ (x2) ಇವುಗಳಲ್ಲಿ **ಬೇರೆ ಬೇರೆ** ಆದಾಗ ಮಾತ್ರ "ಸರಿ (1)". ಇದೇ **XOR** (exclusive OR).

x1 (ಇಡ್ಲಿ)	x2 (ದೋಸೆ)	XOR	ವರ್ಗ
0	0	0	K2
0	1	1	K1
1	0	1	K1
1	1	0	K2



ಚಿತ್ರ ೪: ಹಸಿರು (1) ಬಿಂದುಗಳು ಕರ್ಣಾಕಾರದಲ್ಲಿ, ಕೆಂಪು (0) ಬಿಂದುಗಳು ಇನ್ನೊಂದು ಕರ್ಣದಲ್ಲಿವೆ

ಪ್ರಶ್ನೆ: ಒಂದೇ ಒಂದು ನೇರ ರೇಖೆಯಿಂದ ಹಸಿರು ಮತ್ತು ಕೆಂಪು ಬಿಂದುಗಳನ್ನು ಬೇರ್ಪಡಿಸಲು ಸಾಧ್ಯವೇ?

ಉತ್ತರ: ಸಾಧ್ಯವಿಲ್ಲ! ಯಾವ ಕಡೆ ರೇಖೆ ಎಳೆದರೂ ಕನಿಷ್ಠ ಒಂದು ಬಿಂದು ತಪ್ಪು ಬದಿಯಲ್ಲಿ ಬೀಳುತ್ತದೆ. ಹೀಗಾಗಿ ಒಂದೇ ಪರ್ಸೆಪ್ಟ್ರಾನ್ XOR ಕಲಿಯಲಾರದು. ಈ ಮಿತಿ ಕಂಡುಬಂದಾಗ ಸಂಶೋಧನೆ ಹಲವು ವರ್ಷ ಸ್ಥಗಿತಗೊಂಡಿತು.

ಪರಿಹಾರ: ಮರೆ ಪದರ (hidden layer) ಸೇರಿಸಿ — ಎರಡು ರೇಖೆಗಳು!

"ಒಂದನ್ನು ಮಾತ್ರ" = "ಕನಿಷ್ಠ ಒಂದು ಇದೆ (OR)" ಮತ್ತು "ಎರಡೂ ಇಲ್ಲ (NOT AND)". ಹೀಗೆ ಎರಡು ಸರಳ ನ್ಯೂರಾನ್‌ಗಳನ್ನು ಮರೆ ಪದರದಲ್ಲಿ ಇಟ್ಟು, ಮೂರನೆಯದರಿಂದ ಅವನ್ನು ಸೇರಿಸಬಹುದು:

```
ಮರೆ ನ್ಯೂರಾನ್ H1 (OR ರೇಖೆ) :  H1 = step( x1 + x2 − 0.5 )
ಮರೆ ನ್ಯೂರಾನ್ H2 (AND ರೇಖೆ):  H2 = step( x1 + x2 − 1.5 )
ಔಟ್‌ಪುಟ್ ನ್ಯೂರಾನ್          :  y  = step( H1 − H2 − 0.5 )     (OR ಹೌದು, AND ಇಲ್ಲ)
```

x1	x2	x1+x2−0.5 → H1	x1+x2−1.5 → H2	H1 − H2 − 0.5 → y	ಸರಿಯೇ?
0	0	−0.5 → 0	−1.5 → 0	−0.5 → **0**	✓
0	1	0.5 → 1	−0.5 → 0	0.5 → **1**	✓
1	0	0.5 → 1	−0.5 → 0	0.5 → **1**	✓
1	1	1.5 → 1	0.5 → 1	−0.5 → **0**	✓

> **ದೊಡ್ಡ ಪಾಠ**
>
> ಒಂದು ರೇಖೆ ಸಾಲದಿದ್ದರೆ ಹಲವು ರೇಖೆಗಳನ್ನು ಪದರ-ಪದರವಾಗಿ ಸೇರಿಸಬಹುದು. ಇದೇ **ಬಹು-ಪದರ ಪರ್ಸೆಪ್ಟ್ರಾನ್ (Multi-Layer Perceptron)** — ಇಂದಿನ ಎಲ್ಲ ಡೀಪ್ ಲರ್ನಿಂಗ್‌ನ ಮೂಲ. ಆದರೆ ಹೊಸ ಸಮಸ್ಯೆ: ಮರೆ ಪದರಗಳ ತೂಕಗಳಿಗೆ ಯಾರು ಎಷ್ಟು ತಪ್ಪು ಹೊಣೆ ಎಂದು ಹೇಗೆ ತಿಳಿಯುವುದು? ಅದಕ್ಕೆ ಉತ್ತರವೇ **Backpropagation**.

2. Activation function ಮತ್ತು ಮುಂದಕ್ಕೆ ಸಂಕೇತ ಹರಿವು (Feedforward)

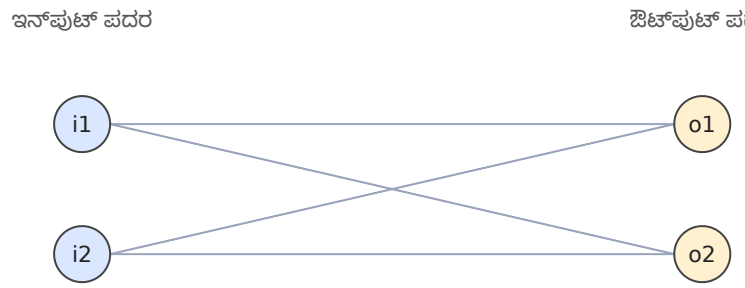
Step ಬದಲು Sigmoid ಏಕೆ?

Step function ಒಮ್ಮೆಲೇ 0 ರಿಂದ 1 ಕ್ಕೆ ಜಿಗಿಯುತ್ತದೆ — ಬೆಳಕಿನ ಸ್ವಿಚ್‌ನಂತೆ (ಆನ್/ಆಫ್). ಆದರೆ ಕಲಿಕೆಗೆ “ಸ್ವಲ್ಪ ಬದಲಾದರೆ ಫಲಿತಾಂಶ ಎಷ್ಟು ಬದಲಾಗುತ್ತದೆ” ಎಂದು ತಿಳಿಯಬೇಕು (ವ್ಯುತ್ಪನ್ನ). ಸ್ವಿಚ್‌ನಲ್ಲಿ ಅದು ಸಾಧ್ಯವಿಲ್ಲ. ಹಾಗಾಗಿ ಮೃದುವಾಗಿ ಏರುವ **Sigmoid** ಬಳಸುತ್ತೇವೆ — ಫ್ಯಾನ್ ರೆಗ್ಯುಲೇಟರ್‌ನಂತೆ (ಕ್ರಮೇಣ ವೇಗ ಹೆಚ್ಚುತ್ತದೆ).

```
Sigmoid:  σ(x) = 1 / (1 + e^(−x))        ಔಟ್‌ಪುಟ್ ಯಾವಾಗಲೂ 0 ಮತ್ತು 1 ರ ನಡುವೆ
```

ಎರಡು-ಪದರದ ಸರಳ ನೆಟ್‌ವರ್ಕ್ (ಹಂತ-ಹಂತದ ಲೆಕ್ಕ)

ಸಂಕೇತ ಕೇವಲ ಒಂದೇ ದಿಕ್ಕಿನಲ್ಲಿ (ಇನ್‌ಪುಟ್ → ಔಟ್‌ಪುಟ್) ಹರಿಯುವ ನೆಟ್‌ವರ್ಕ್‌ಗೆ **Feedforward network** ಎನ್ನುತ್ತಾರೆ.



ಚಿತ್ರ ೫: ಎರಡು ಇನ್‌ಪುಟ್, ಎರಡು ಔಟ್‌ಪುಟ್ ನೋಡ್‌ಗಳು

ಇನ್‌ಪುಟ್‌ಗಳು: i1 = 1.0, i2 = 0.5. ತೂಕಗಳು: w1,1 = 0.9, w2,1 = 0.3, w1,2 = 0.2, w2,2 = 0.8.

ಔಟ್‌ಪುಟ್ ನೋಡ್ 1

```
ಹಂತ ೧: ಮೊತ್ತ  x = (1.0 × 0.9) + (0.5 × 0.3) = 0.9 + 0.15 = 1.05
ಹಂತ ೨: Sigmoid y = 1 / (1 + e^(−1.05)) = 0.7408
```

ಔಟ್‌ಪುಟ್ ನೋಡ್ 2

```
ಹಂತ ೧: ಮೊತ್ತ  x = (1.0 × 0.2) + (0.5 × 0.8) = 0.2 + 0.4 = 0.6
ಹಂತ ೨: Sigmoid y = 1 / (1 + e^(−0.6)) = 0.6457
```

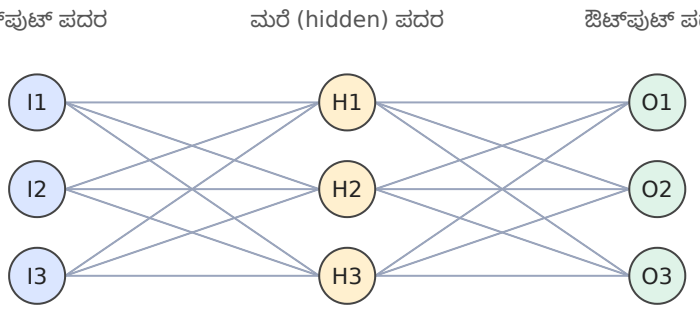
ಅದೇ ಲೆಕ್ಕ ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನಲ್ಲಿ

ನೋಡ್‌ಗಳು ಸಾವಿರಾರು ಇದ್ದಾಗ ಒಂದೊಂದಾಗಿ ಲೆಕ್ಕ ಹಾಕುವುದು ಅಸಾಧ್ಯ. ಎಲ್ಲವನ್ನೂ ಒಂದೇ ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರದಲ್ಲಿ ಮಾಡುತ್ತೇವೆ: `X = W · I`, ನಂತರ `O = σ(X)`.

```
| 0.9  0.3 |   | 1.0 |     | 0.9×1.0 + 0.3×0.5 |     | 1.05 |
| 0.2  0.8 | · | 0.5 |  =  | 0.2×1.0 + 0.8×0.5 |  =  | 0.60 |     →  σ →
[0.7408, 0.6457]
```

೮. ಮೂರು ಪದರದ ನೆಟ್‌ವರ್ಕ್: ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನಿಂದ ಲೆಕ್ಕ

ಇನ್‌ಪುಟ್ → **ಮರೆ (hidden) ಪದರ** → ಔಟ್‌ಪುಟ್. ಪ್ರತಿ ಪದರಕ್ಕೂ ಇದೇ ಎರಡು ಹಂತ: ಮ್ಯಾಟ್ರಿಕ್ಸ್ ಗುಣಾಕಾರ, ನಂತರ sigmoid.



ಚಿತ್ರ ೬: ಮೂರು–ಮೂರು–ಮೂರು ನೋಡ್‌ಗಳ ನೆಟ್‌ವರ್ಕ್

ಇನ್‌ಪುಟ್ ಮತ್ತು ತೂಕಗಳು

```
I = [0.9, 0.1, 0.8]

W(ಇನ್‌ಪುಟ್→ಮರೆ) = | 0.9  0.3  0.4 |      W(ಮರೆ→ಔಟ್‌ಪುಟ್) = | 0.3  0.7  0.5 |
                    | 0.2  0.8  0.2 |                          | 0.6  0.5
0.2 |
                    | 0.1  0.5  0.6 |                          | 0.8  0.1
0.9 |
```

ಹಂತ ೧: ಮರೆ ಪದರ

```
X_hidden = W · I:
  ನೋಡ್ 1: 0.9×0.9 + 0.3×0.1 + 0.4×0.8 = 0.81 + 0.03 + 0.32 = 1.16
  ನೋಡ್ 2: 0.2×0.9 + 0.8×0.1 + 0.2×0.8 = 0.18 + 0.08 + 0.16 = 0.42
  ನೋಡ್ 3: 0.1×0.9 + 0.5×0.1 + 0.6×0.8 = 0.09 + 0.05 + 0.48 = 0.62

O_hidden = σ(X_hidden) = [σ(1.16), σ(0.42), σ(0.62)] = [0.761, 0.603, 0.650]
```

ಹಂತ ೨: ಔಟ್‌ಪುಟ್ ಪದರ

```
X_output = W · O_hidden:
  ನೋಡ್ 1: 0.3×0.761 + 0.7×0.603 + 0.5×0.650 = 0.228 + 0.422 + 0.325 = 0.975
  ನೋಡ್ 2: 0.6×0.761 + 0.5×0.603 + 0.2×0.650 = 0.457 + 0.302 + 0.130 = 0.888
  ನೋಡ್ 3: 0.8×0.761 + 0.1×0.603 + 0.9×0.650 = 0.609 + 0.060 + 0.585 = 1.254

O_output = σ(X_output) = [σ(0.975), σ(0.888), σ(1.254)] = [0.726, 0.708,
0.778]
```

ಫಲಿತಾಂಶ

ಇನ್‌ಪುಟ್ [0.9, 0.1, 0.8] ಗೆ ನೆಟ್‌ವರ್ಕ್‌ನ ಊಹೆ **[0.726, 0.708, 0.778]**. ಇದು ಸರಿಯಾದ ಉತ್ತರದಿಂದ ಎಷ್ಟು ದೂರ ಎಂದು ಈಗ ಅಳೆಯಬೇಕು.

೯. ತಪ್ಪನ್ನು (Error) ಅಳೆಯುವುದು: ಶಿಕ್ಷಕರೊಂದಿಗೆ ಕಲಿಕೆ

Supervised learning (ಶಿಕ್ಷಕರೊಂದಿಗೆ ಕಲಿಕೆ) ಎಂದರೆ ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಪ್ರತಿ ಉದಾಹರಣೆಯ ಜೊತೆಗೆ ಸರಿಯಾದ ಉತ್ತರವನ್ನೂ ಕೊಡುವುದು. ಶಾಲೆಯಲ್ಲಿ ಶಿಕ್ಷಕರು ಪ್ರಶ್ನೆಯ ಜೊತೆ ಉತ್ತರ ಹೇಳಿಕೊಟ್ಟು, ನಂತರ ನಮ್ಮ ಉತ್ತರ ತಿದ್ದುವಂತೆ.

ಪದ	ಸಂಕೇತ	ಅರ್ಥ
Input (ಇನ್‌ಪುಟ್)	I	ನೆಟ್‌ವರ್ಕ್‌ಗೆ ಕೊಡುವ ಸಂಖ್ಯೆಗಳ ಸಾಲು (features)
Target / Label	t	ಸರಿಯಾದ ಉತ್ತರ — ಶಿಕ್ಷಕರ ಉತ್ತರ
Predicted output	o	ನೆಟ್‌ವರ್ಕ್ ಕೊಟ್ಟ ಉತ್ತರ — ವಿದ್ಯಾರ್ಥಿಯ ಉತ್ತರ
Error / Loss	$e = t - o$	ಇವೆರಡರ ವ್ಯತ್ಯಾಸ
Learning rate	η ಅಥವಾ α	ತೂಕ ಬದಲಿಸುವ ಹೆಜ್ಜೆಯ ಗಾತ್ರ (ಚಿಕ್ಕ ಸಂಖ್ಯೆ)

ತಪ್ಪನ್ನು ಅಳೆಯುವ ಮೂರು ವಿಧಾನ

ಮೂರು ಉದಾಹರಣೆಗಳಲ್ಲಿ ನೆಟ್‌ವರ್ಕ್ ಊಹೆ (o) ಮತ್ತು ಸರಿಯಾದ ಉತ್ತರ (t):

o (ಊಹೆ)	t (ಸರಿಯಾದ)	$t - o$	$\lvert t - o \rvert$	$(t - o)^2$
0.4	0.5	0.1	0.1	0.01
0.8	0.7	−0.1	0.1	0.01
1.0	1.0	0	0	0
ಮೊತ್ತ		**0**	**0.2**	**0.02**

> **ಎಚ್ಚರ!**
> ಸರಳ ವ್ಯತ್ಯಾಸ $(t - o)$ ಕೂಡಿದರೆ +0.1 ಮತ್ತು −0.1 ಒಂದನ್ನೊಂದು ರದ್ದು ಮಾಡಿ ಮೊತ್ತ **0** ಬರುತ್ತದೆ — ನೆಟ್‌ವರ್ಕ್ ತಪ್ಪು ಮಾಡಿದ್ದರೂ "ಏನೂ ತಪ್ಪಿಲ್ಲ" ಎನಿಸುತ್ತದೆ! ಅದಕ್ಕೆ ಎರಡು ಪರಿಹಾರ:

- **MAE (Mean Absolute Error)** = ಸರಾಸರಿ $|t - o| = 0.2 / 3 \approx$ **0.067**
- **MSE (Mean Squared Error)** = ಸರಾಸರಿ $(t - o)^2 = 0.02 / 3 \approx$ **0.0067**. ವರ್ಗ ಮಾಡುವುದರಿಂದ ಋಣ ಚಿಹ್ನೆ ಹೋಗುತ್ತದೆ, ಮತ್ತು ದೊಡ್ಡ ತಪ್ಪಿಗೆ ದೊಡ್ಡ ಶಿಕ್ಷೆ ಸಿಗುತ್ತದೆ. ಜೊತೆಗೆ ಇದು ಸುಲಭವಾಗಿ ವ್ಯುತ್ಪನ್ನ ಮಾಡಬಹುದಾದ ನಯವಾದ ವಕ್ರ — ಆದ್ದರಿಂದ ಈ ಉಪನ್ಯಾಸದಲ್ಲಿ ಇದನ್ನೇ ಬಳಸುತ್ತೇವೆ.

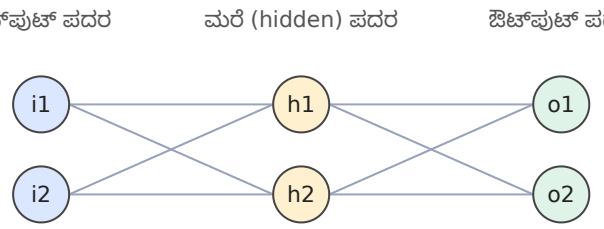
೧೦. ತಪ್ಪನ್ನು ಹಿಂದಕ್ಕೆ ಹಂಚುವುದು (Backpropagation ನ ಮೂಲ ಕಲ್ಪನೆ)

ತೂಕದ ಎರಡು ಪಾತ್ರಗಳು

- **ಮುಂದಕ್ಕೆ (Forward):** ತೂಕಗಳು ಸಂಕೇತವನ್ನು ಇನ್‌ಪುಟ್‌ನಿಂದ ಔಟ್‌ಪುಟ್‌ಗೆ ಕೊಂಡೊಯ್ಯುತ್ತವೆ.
- **ಹಿಂದಕ್ಕೆ (Backward):** ಅದೇ ತೂಕಗಳು ತಪ್ಪನ್ನು ಔಟ್‌ಪುಟ್‌ನಿಂದ ಹಿಂದಕ್ಕೆ ಹಂಚುತ್ತವೆ.

> **ಕಥೆ: ಗುಂಪು ಪ್ರಾಜೆಕ್ಟ್ ಮತ್ತು ಕಡಿಮೆ ಅಂಕ**
>
> ನಾಲ್ವರು ಗೆಳೆಯರು ಸೇರಿ ಪ್ರಾಜೆಕ್ಟ್ ಮಾಡಿದರು; ಶಿಕ್ಷಕರು 10 ಅಂಕ ಕಡಿಮೆ ಕೊಟ್ಟರು. ಈ ನಷ್ಟಕ್ಕೆ ಯಾರು ಎಷ್ಟು ಕಾರಣ? ಹೆಚ್ಚು ಕೆಲಸ ಮಾಡಿದವನ ಕೊಡುಗೆ ಹೆಚ್ಚಿತ್ತು, ಹಾಗಾಗಿ ತಪ್ಪಿನ ದೊಡ್ಡ ಪಾಲು ಅವನಿಗೆ: **ಪ್ರಭಾವಕ್ಕೆ ಅನುಗುಣವಾಗಿ ತಪ್ಪನ್ನು ಹಂಚುವುದು** — ಬ್ಯಾಕ್‌ಪ್ರೊಪಗೇಶನ್‌ನ ಆತ್ಮ ಇದೇ. ಸಮಾನವಾಗಿ ಹಂಚುವುದು (½, ½) ಸುಲಭ, ಆದರೆ ತೂಕದ ಪ್ರಭಾವದ ಅನುಪಾತದಲ್ಲಿ (¾, ¼) ಹಂಚುವುದು ಹೆಚ್ಚು ನ್ಯಾಯ.

ಉದಾಹರಣೆ: ಎರಡು ಔಟ್‌ಪುಟ್ ನ್ಯೂರಾನ್‌ಗಳ ತಪ್ಪು ಮರೆ ಪದರಕ್ಕೆ



ಚಿತ್ರ ೭: ಹಿಂದಕ್ಕೆ ಹರಿಯುವ ತಪ್ಪು

ಔಟ್‌ಪುಟ್ ತಪ್ಪುಗಳು: e1 = 0.8, e2 = 0.5. ತೂಕಗಳು: w11 = 2, w21 = 3 (ಇವು ಔಟ್‌ಪುಟ್ 1 ಗೆ ಬರುತ್ತವೆ), w12 = 1, w22 = 4 (ಇವು ಔಟ್‌ಪುಟ್ 2 ಗೆ ಬರುತ್ತವೆ).

ಹಂತ ೧: ಪ್ರತಿ ಔಟ್‌ಪುಟ್‌ನ ತಪ್ಪನ್ನು ಅದರ ತೂಕಗಳ ಅನುಪಾತದಲ್ಲಿ ವಿಭಜಿಸಿ

ಔಟ್‌ಪುಟ್ 1 ರ e1 = 0.8: w11 ಗೆ ಪಾಲು = 2/(2+3) = 0.4, w21 ಗೆ ಪಾಲು = 3/(2+3) = 0.6

ಔಟ್‌ಪುಟ್ 2 ರ e2 = 0.5: w12 ಗೆ ಪಾಲು = 1/(1+4) = 0.2, w22 ಗೆ ಪಾಲು = 4/(1+4) = 0.8

ಹಂತ ೨: ಪ್ರತಿ ಮರೆ ನೋಡ್‌ಗೆ ಬಂದ ಪಾಲುಗಳನ್ನು ಕೂಡಿ

```
e_hidden,1 = 0.8 × 0.4 + 0.5 × 0.2 = 0.32 + 0.10 = 0.42
e_hidden,2 = 0.8 × 0.6 + 0.5 × 0.8 = 0.48 + 0.40 = 0.88
```

ಅಂದರೆ ಮರೆ ನೋಡ್ 2 ಹೆಚ್ಚು ಹೊಣೆ (0.88) — ಅದರ ತೂಕಗಳು ಹೆಚ್ಚು ಬದಲಾಗಬೇಕು.

ಮ್ಯಾಟ್ರಿಕ್ಸ್ ರೂಪ (ಸರಳೀಕರಣ)

ಭಾಗಾಕಾರದ (ಅನುಪಾತ) ಭಾಗ ಬಿಟ್ಟರೂ ತಪ್ಪುಗಳ ಅನುಪಾತ ಅದೇ ಉಳಿಯುತ್ತದೆ, ಹಾಗಾಗಿ ನೇರವಾಗಿ ತೂಕ ಮ್ಯಾಟ್ರಿಕ್ಸ್‌ನ **ವಿಲೋಮ (transpose)** ಬಳಸುತ್ತೇವೆ: `error_hidden = Wᵀ · error_output`.

```
error_hidden = | w11  w12 | · | e1 | = | 2×0.8 + 1×0.5 | = | 2.1 |
               | w21  w22 |   | e2 |   | 3×0.8 + 4×0.5 |   | 4.4 |
```

ಇಲ್ಲಿ ಭಾಗಾಕಾರದ ಛೇದ (2+3 = 1+4 = 5) ಎರಡಕ್ಕೂ ಸಮಾನವಾದ್ದರಿಂದ 2.1/5 = 0.42 ಮತ್ತು 4.4/5 = 0.88 — ಹಿಂದಿನ ಉತ್ತರವೇ, ಕೇವಲ ಒಂದು ಸ್ಥಿರ ಅಂಶದ ವ್ಯತ್ಯಾಸ. ಇದೇ ಮ್ಯಾಟ್ರಿಕ್ಸ್ ವಿಧಾನವನ್ನು ನಿಜವಾದ ನೆಟ್‌ವರ್ಕ್‌ಗಳಲ್ಲಿ ಬಳಸುತ್ತಾರೆ.

೧೧. ತೂಕ ಬದಲಿಸುವ ಸರಿಯಾದ ದಾರಿ: Gradient Descent

ತಪ್ಪು ತಿಳಿಯಿತು. ಈಗ ತೂಕಗಳನ್ನು **ಎಷ್ಟು ಮತ್ತು ಯಾವ ದಿಕ್ಕಿಗೆ** ಬದಲಿಸಬೇಕು? ಮೂರು ದಾರಿಗಳನ್ನು ಯೋಚಿಸೋಣ:

ದಾರಿ	ಸಮಸ್ಯೆ
೧. ಸಮೀಕರಣವನ್ನು ಬಿಡಿಸಿ ನೇರ ಉತ್ತರ ಪಡೆಯುವುದು	ಕಷ್ಟ — ಫಂಕ್ಷನ್‌ಗಳು ಒಂದರೊಳಗೊಂದು ಸಿಕ್ಕಿಕೊಂಡಿವೆ (sigmoid ಒಳಗೆ sigmoid…)
೨. ಎಲ್ಲ ಸಂಯೋಜನೆಗಳನ್ನು ಪ್ರಯತ್ನಿಸುವುದು (brute force)	ಪ್ರತಿ ತೂಕ −1 ರಿಂದ 1 ರ ನಡುವೆ 1000 ಮೌಲ್ಯ ತೆಗೆದುಕೊಳ್ಳಲಿ. 18 ತೂಕಗಳ ಸಣ್ಣ ನೆಟ್‌ವರ್ಕ್‌ಗೆ $1000^{18} = \mathbf{10^{54}}$ ಸಂಯೋಜನೆ — ವಿಶ್ವದ ಆಯುಷ್ಯದಲ್ಲೂ ಸಾಧ್ಯವಿಲ್ಲ!
೩. Gradient Descent	**ಹೆಜ್ಜೆ ಹೆಜ್ಜೆಯಾಗಿ ತಪ್ಪು ಕಡಿಮೆ ಆಗುವ ದಿಕ್ಕಿಗೆ ಹೋಗುವುದು — ಇದೇ ಸರಿಯಾದ ದಾರಿ**

> **ಕಥೆ: ಕತ್ತಲಲ್ಲಿ ಬೆಟ್ಟದಿಂದ ಕಣಿವೆಗೆ**
> ನೀವು ನಂದಿ ಬೆಟ್ಟದ ಮೇಲೆ ಮಂಜಿನಲ್ಲಿ ಸಿಕ್ಕಿಕೊಂಡಿದ್ದೀರಿ; ಕೆಳಗೆ ಊರಿಗೆ ಇಳಿಯಬೇಕು, ಆದರೆ ಏನೂ ಕಾಣುವುದಿಲ್ಲ. ಏನು ಮಾಡುವಿರಿ? ಕಾಲಿನ ಕೆಳಗೆ ನೆಲ ಯಾವ ಕಡೆ ಹೆಚ್ಚು ಇಳಿಜಾರಾಗಿದೆ ಎಂದು ಸ್ಪರ್ಶಿಸಿ, ಆ ಕಡೆ **ಒಂದು ಸಣ್ಣ ಹೆಜ್ಜೆ** ಇಡುತ್ತೀರಿ. ಮತ್ತೆ ಸ್ಪರ್ಶಿಸಿ, ಮತ್ತೆ ಹೆಜ್ಜೆ. ಹೀಗೇ ಮುಂದುವರಿದರೆ ಕೊನೆಗೆ ಕಣಿವೆ ತಲುಪುತ್ತೀರಿ. ಇದೇ ಗ್ರೇಡಿಯಂಟ್ ಡಿಸೆಂಟ್.

ಗ್ರೇಡಿಯಂಟ್ ಎಂದರೇನು?

ಕಾರ್ಯದ (ಇಲ್ಲಿ ತಪ್ಪಿನ) ಪ್ರತಿ ಚರಾಂಕದ ಭಾಗಶಃ ವ್ಯುತ್ಪನ್ನಗಳ ಸದಿಶವೇ **ಗ್ರೇಡಿಯಂಟ್ (∇)**. ಅದು **ತಪ್ಪು ಅತಿ ವೇಗವಾಗಿ ಹೆಚ್ಚುವ ದಿಕ್ಕನ್ನು** ತೋರಿಸುತ್ತದೆ. ನಮಗೆ ತಪ್ಪು ಕಡಿಮೆ ಮಾಡಬೇಕು, ಹಾಗಾಗಿ ಅದರ **ವಿರುದ್ಧ ದಿಕ್ಕಿಗೆ** (−∇) ಹೋಗುತ್ತೇವೆ.

Gradient Descent ಸೂತ್ರ

```
x_ಹೊಸ = x_ಹಳೆಯ − λ × ∇f(x_ಹಳೆಯ)

f  = ಕನಿಷ್ಠಗೊಳಿಸಬೇಕಾದ ತಪ್ಪಿನ ಫಂಕ್ಷನ್ (loss)
∇f = ಗ್ರೇಡಿಯಂಟ್ (ಏರುವ ದಿಕ್ಕು),  "−" ಚಿಹ್ನೆ = ಇಳಿಯುವ ದಿಕ್ಕು
λ  = ಹೆಜ್ಜೆಯ ಗಾತ್ರ (learning rate)
```

ಅಲ್ಗೊರಿದಮ್: ೧) ಆರಂಭಿಕ ಬಿಂದು ಆಯ್ಕೆ. ೨) ಹೆಜ್ಜೆಯ ಗಾತ್ರ ನಿರ್ಧಾರ (ಸ್ಥಿರ / ಬದಲಾಗುವ / line search). ೩) ಸೂತ್ರದಿಂದ ಹೊಸ ಬಿಂದು. ೪) ಬದಲಾವಣೆ ಅತ್ಯಲ್ಪವಾದರೆ ಅಥವಾ ಗರಿಷ್ಠ ಸುತ್ತು ತಲುಪಿದರೆ ನಿಲ್ಲಿಸು, ಇಲ್ಲದಿದ್ದರೆ ೩ ಕ್ಕೆ ಹಿಂತಿರುಗು.

ಹಂತ-ಹಂತದ ಉದಾಹರಣೆ: $z = x^2 - x + y^2 - 2y$ ರ ಕನಿಷ್ಠ ಕಂಡುಹಿಡಿಯಿರಿ

ಹಂತ ೧: ಗ್ರೇಡಿಯಂಟ್

$\partial z/\partial x = 2x - 1$, $\partial z/\partial y = 2y - 2$. ಆರಂಭ ಬಿಂದು A(0, 0): ಗ್ರೇಡಿಯಂಟ್ = (−1, −2), z(A) = 0.

ಹಂತ ೨: ಗ್ರೇಡಿಯಂಟ್‌ನ ವಿರುದ್ಧ ದಿಕ್ಕಿನಲ್ಲಿ ರೇಖೆ

$x = 0 + (-1)\cdot t$, $y = 0 + (-2)\cdot t$. ಇಳಿಯಲು t ಋಣ ಸಂಖ್ಯೆ ಆಗಿರಬೇಕು (ಸೂತ್ರದಲ್ಲಿ "+" ಇದ್ದರೂ ಋಣ t ಇಂದ ವಿರುದ್ಧ ದಿಕ್ಕಿಗೆ ಹೋಗುತ್ತದೆ).

ಹಂತ ೩: $t = -1$ ಪ್ರಯತ್ನ

B = (1, 2): $z(B) = 1 - 1 + 4 - 4 = \mathbf{0}$ — z ಬದಲಾಗಲಿಲ್ಲ! ಕನಿಷ್ಠ ಈ ಎರಡು ಬಿಂದುಗಳ ನಡುವೆ ಇರಬಹುದು.

ಹಂತ ೪: ಚಿಕ್ಕ ಹೆಜ್ಜೆ $t = -0.5$

C = (0.5, 1): $z(C) = 0.25 - 0.5 + 1 - 2 = \mathbf{-1.25}$ (ಕಡಿಮೆ!). C ನಲ್ಲಿ ಗ್ರೇಡಿಯಂಟ್: $2(0.5) - 1 = 0$ ಮತ್ತು $2(1) - 2 = 0$ → ಎರಡೂ ಸೊನ್ನೆ.

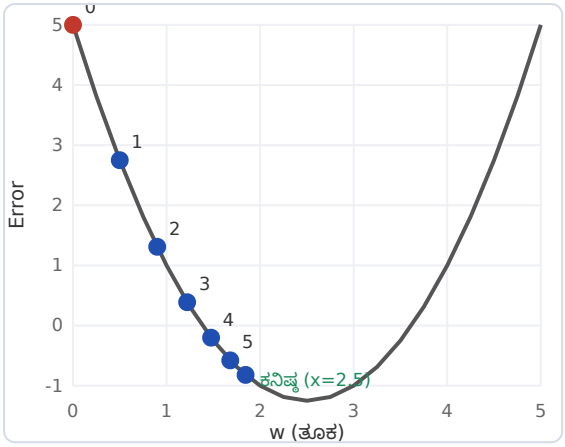
ತೀರ್ಮಾನ

ಗ್ರೇಡಿಯಂಟ್ ಸೊನ್ನೆ ಎಂದರೆ ಇಳಿಜಾರೇ ಇಲ್ಲ — ನಾವು ಕಣಿವೆಯ ತಳ ತಲುಪಿದ್ದೇವೆ. ಕನಿಷ್ಠ ಬಿಂದು **(0.5, 1)**, ಕನಿಷ್ಠ ಮೌಲ್ಯ **−1.25**.

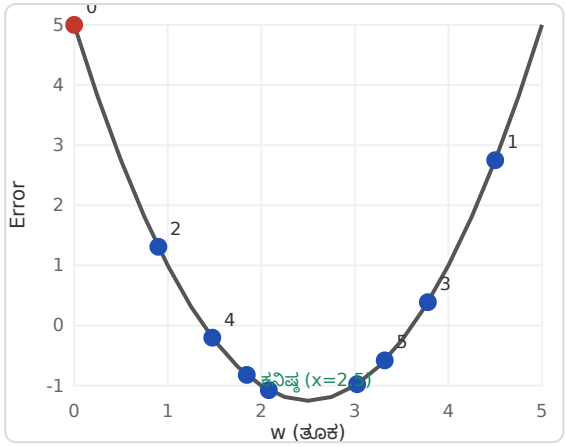
೧೨. Learning rate, ಸ್ಥಳೀಯ ಕನಿಷ್ಠ, Vanishing / Exploding gradients

Learning rate ಎಷ್ಟಿರಬೇಕು?

ಒಂದೇ ತೂಕ ಇರುವ ಸರಳ ಉದಾಹರಣೆ: ತಪ್ಪು $E(w) = w^2 - 5w + 5$ (ಕನಿಷ್ಠ w = 2.5 ರಲ್ಲಿ, ಮೌಲ್ಯ −1.25). ಗ್ರೇಡಿಯಂಟ್ = $2w - 5$. ಆರಂಭ $w = 0$. ನಿಯಮ: $w_{\text{ಹೊಸ}} = w - \eta\cdot(2w - 5)$.



ಚಿತ್ರ ೮: η = 0.1 — ಸ್ಥಿರವಾಗಿ ಕನಿಷ್ಠದ ಕಡೆ ಇಳಿಯುತ್ತದೆ



ಚಿತ್ರ ೯: η = 0.9 — ಕನಿಷ್ಠದ ಎರಡೂ ಬದಿ ಹಾರುತ್ತಾ ಹತ್ತಿರವಾಗುತ್ತದೆ

ಪ್ರಕರಣ	ಸುತ್ತುಗಳು (N)	η	ಅಂತಿಮ w	E(w)	ಏನಾಯಿತು?
೧	5	0.05	1.02	0.93	ತುಂಬ ನಿಧಾನ — ಇನ್ನೂ ದೂರದಲ್ಲಿದೆ
೨	15	0.1	2.41	−1.24	**ಚೆನ್ನಾಗಿ ತಲುಪಿತು (ಗುರಿ 2.5)**
೩	5	1.0	5.0	5.0	**ಹಾರುತ್ತಲೇ ಇದೆ (0 ↔ 5), ಎಂದಿಗೂ ನೆಲೆಸುವುದಿಲ್ಲ**
೪	15	0.9	2.59	−1.24	ದೊಡ್ಡ ಹೆಜ್ಜೆ, ಆದರೂ ಅಲುಗಾಡುತ್ತಾ ತಲುಪಿತು

> **ನಿಯಮ**
>
> ತುಂಬ ಚಿಕ್ಕ η → ಕಲಿಕೆ ಆಮೆ ವೇಗ. ತುಂಬ ದೊಡ್ಡ η → ಕಣಿವೆಯನ್ನು ದಾಟಿ ಹಾರಿ ಹೋಗುತ್ತದೆ (ಬಸ್‌ನಲ್ಲಿ ನಿಲ್ದಾಣ ತಪ್ಪಿಸಿಕೊಳ್ಳುವಂತೆ). ಸರಿಯಾದ ಮಧ್ಯದ ಮೌಲ್ಯವನ್ನು ಪ್ರಯೋಗದಿಂದ ಆರಿಸಬೇಕು.

ಸ್ಥಳೀಯ ಕನಿಷ್ಠ ಮತ್ತು ಸಮತಟ್ಟು (Local minimum, Plateau)

ನಿಜವಾದ ನೆಟ್‌ವರ್ಕ್‌ನ ತಪ್ಪಿನ ಮೇಲ್ಮೈ ಒಂದೇ ಬಟ್ಟಲಲ್ಲ — ಬೆಟ್ಟ-ಗುಡ್ಡಗಳ ಭೂಪ್ರದೇಶದಂತೆ. **Local minimum** ಎಂದರೆ ಸುತ್ತಮುತ್ತ ಅತಿ ಕೆಳಗಿನ ಬಿಂದು ಆದರೆ ಒಟ್ಟಾರೆ ಅತಿ ಕೆಳಗಿನದಲ್ಲ (**global minimum**). **Plateau** ಎಂದರೆ ಸಮತಟ್ಟಾದ ಜಾಗ; ಅಲ್ಲಿ ಗ್ರೇಡಿಯಂಟ್ ಬಹುತೇಕ ಸೊನ್ನೆಯಾಗಿ ಕಲಿಕೆ ನಿಧಾನವಾಗುತ್ತದೆ. ಮಂಜಿನಲ್ಲಿ ಕಣಿವೆ ಎಂದು ತಿಳಿದು ಒಂದು ಹೊಂಡದಲ್ಲಿ ಇಳಿದು ನಿಲ್ಲುವಂತೆ.

Vanishing ಮತ್ತು Exploding gradients

ಸಮಸ್ಯೆ	ಏನಾಗುತ್ತದೆ?	ಪರಿಣಾಮ
Vanishing (ಮಾಯವಾಗುವ)	ಹಿಂದಕ್ಕೆ ಹೋಗುತ್ತಾ ಗ್ರೇಡಿಯಂಟ್ ಪದರದಿಂದ ಪದರಕ್ಕೆ ಚಿಕ್ಕದಾಗುತ್ತಾ ಹೋಗುತ್ತದೆ	ಆರಂಭದ ಪದರಗಳು ಬಹುತೇಕ ಕಲಿಯುವುದೇ ಇಲ್ಲ
Exploding (ಸ್ಫೋಟಿಸುವ)	ಗ್ರೇಡಿಯಂಟ್ ಬೃಹತ್ತಾಗಿ ಬೆಳೆಯುತ್ತದೆ	ತೂಕಗಳು ಅಗಾಧವಾಗಿ, ಕೊನೆಗೆ NaN ಆಗಿ ಮಾದರಿ ಅಸ್ಥಿರ

ಏಕೆ ಹೀಗಾಗುತ್ತದೆ? — ಸಂಖ್ಯೆಗಳಲ್ಲಿ

Sigmoid ನ ವ್ಯುತ್ಪನ್ನ σ(x)(1 − σ(x)) ಗರಿಷ್ಠ ಮೌಲ್ಯ **0.25**. ಚೈನ್ ರೂಲ್‌ನಲ್ಲಿ ಪ್ರತಿ ಪದರದಲ್ಲಿ ಇದನ್ನು ಗುಣಿಸುತ್ತಾ ಹೋದರೆ:

```
5 ಪದರ  : 0.25⁵  = 0.00098    (ಸುಮಾರು 1000 ರಲ್ಲಿ 1 ಪಾಲು ಉಳಿಯಿತು)
10 ಪದರ : 0.25¹⁰ = 0.00000095 (ಬಹುತೇಕ ಸೊನ್ನೆ — ಮೊದಲ ಪದರ ಕಲಿಯುವುದೇ ಇಲ್ಲ)
```

ಹತ್ತು ಜನರ ಸರಪಳಿಯಲ್ಲಿ ಪ್ರತಿಯೊಬ್ಬರೂ ಸಂದೇಶದ ಕಾಲು ಭಾಗ ಮಾತ್ರ ಮುಂದಕ್ಕೆ ಹೇಳಿದರೆ ಕೊನೆಯವನಿಗೆ ಏನೂ ತಲುಪುವುದಿಲ್ಲ — ಹಾಗೆಯೇ. ವಿರುದ್ಧವಾಗಿ ಪ್ರತಿ ಹಂತದಲ್ಲಿ ಗುಣಕ 1 ಕ್ಕಿಂತ ದೊಡ್ಡದಿದ್ದರೆ (ಉದಾ: 2) ಸಂಖ್ಯೆ ಸ್ಫೋಟಿಸುತ್ತದೆ ($2^{10} = 1024$).

೧೩. Backpropagation ಸೂತ್ರದ ಉತ್ಪತ್ತಿ ಮತ್ತು ತೂಕದ ನಿಜವಾದ ಬದಲಾವಣೆ

ಇತಿಹಾಸ

- **೧೯೭೦:** ಸೆಪ್ಪೊ ಲಿನ್ನೈನ್ಮಾ (Seppo Linnainmaa) ಚೈನ್ ರೂಲ್ ಬಳಸಿ ಗ್ರೇಡಿಯಂಟ್‌ಗಳನ್ನು ತಾನಾಗಿ ಲೆಕ್ಕ ಹಾಕುವ ವಿಧಾನ ವಿವರಿಸಿದರು.
- **೧೯೮೬:** ರುಮೆಲ್‌ಹಾರ್ಟ್, ಹಿಂಟನ್ ಮತ್ತು ವಿಲಿಯಮ್ಸ್ ಇದನ್ನು ಬಹು-ಪದರ ನೆಟ್‌ವರ್ಕ್‌ಗಳ ತರಬೇತಿಗೆ ಪ್ರಾಯೋಗಿಕವಾಗಿ ಜನಪ್ರಿಯಗೊಳಿಸಿದರು — ಇಲ್ಲಿಂದ ಡೀಪ್ ಲರ್ನಿಂಗ್ ಯುಗ ಶುರುವಾಯಿತು.

ಸೂತ್ರ ಹೇಗೆ ಬರುತ್ತದೆ? (ಒಂದು ತೂಕ w_{jk} ಗೆ)

ತಪ್ಪು $E = (t_k - o_k)^2$. ಈ ತಪ್ಪನ್ನು ತೂಕ w_{jk} ಹೇಗೆ ಪ್ರಭಾವಿಸುತ್ತದೆ? ಸರಪಳಿ: **w → (ಮೊತ್ತ) → σ → o → E**. ಚೈನ್ ರೂಲ್‌ನಿಂದ ಪ್ರತಿ ಕೊಂಡಿಯ ವ್ಯುತ್ಪನ್ನ ಗುಣಿಸಿ:

```
ಹಂತ ೧: dE/do_k   = −2 (t_k − o_k)                    (ವರ್ಗದ ವ್ಯುತ್ಪನ್ನ)
ಹಂತ ೨: do_k/dz   = σ(z)(1 − σ(z))                    (sigmoid ನ ವ್ಯುತ್ಪನ್ನ), z = Σ w_jk·o_j
ಹಂತ ೩: dz/dw_jk  = o_j                               (ತೂಕದ ಇನ್‌ಪುಟ್)

ಗುಣಿಸಿ:  dE/dw_jk = −2 (t_k − o_k) · σ(Σ w_jk·o_j) · (1 − σ(Σ w_jk·o_j)) · o_j

(ಸ್ಥಿರ 2 ಅನ್ನು learning rate ಜೊತೆ ಸೇರಿಸಿ ತೆಗೆದರೆ)
         dE/dw_jk = −(t_k − o_k) · σ(…) · (1 − σ(…)) · o_j
```

ತೂಕ ಬದಲಾವಣೆ ಸೂತ್ರ

```
w_ಹೊಸ = w_ಹಳೆಯ − α × dE/dw          α = learning rate
```

ಮರೆ ಪದರಕ್ಕೆ: ಅದೇ ಸೂತ್ರ, ಆದರೆ $(t_k - o_k)$ ಬದಲು ಹಿಂದಕ್ಕೆ ಹಂಚಿದ ತಪ್ಪು e_j (ಭಾಗ ೧೦), ಮತ್ತು o_j ಬದಲು ಇನ್‌ಪುಟ್ ಪದರದ ಸಂಕೇತ o_i.

ನಿಜವಾದ ಸಂಖ್ಯೆಗಳಲ್ಲಿ: w11 ಅನ್ನು ಬದಲಿಸೋಣ

ಔಟ್‌ಪುಟ್ ನೋಡ್ 1 ರ ತಪ್ಪು e1 = t − o = 0.8. ಮರೆ ನೋಡ್‌ಗಳ ಔಟ್‌ಪುಟ್‌ಗಳು $o_1 = 0.4$, $o_2 = 0.5$. ತೂಕಗಳು w11 = 2.0, w21 = 3.0. α = 0.1.

```
ಹಂತ ೧: ಮೊತ್ತ          x = Σ w_j1·o_j = (2.0×0.4) + (3.0×0.5) = 0.8 + 1.5 = 2.3
ಹಂತ ೨: Sigmoid        σ(2.3) = 1 / (1 + e^(−2.3)) = 0.909
ಹಂತ ೩: ವ್ಯುತ್ಪನ್ನ     σ(1 − σ) = 0.909 × (1 − 0.909) = 0.083
ಹಂತ ೪: o_j            w11 ಗೆ ಸಂಬಂಧಿಸಿದ ಇನ್‌ಪುಟ್ = o_1 = 0.4
ಹಂತ ೫: ಗ್ರೇಡಿಯಂಟ್      dE/dw11 = −0.8 × 0.083 × 0.4 = −0.0266
ಹಂತ ೬: ಹೊಸ ತೂಕ       w11 = 2.0 − 0.1 × (−0.0266) = 2.0 + 0.00266 = 2.0027
```

ಅರ್ಥ: ಗ್ರೇಡಿಯಂಟ್ ಋಣ → w11 ಹೆಚ್ಚಿಸಿದರೆ ತಪ್ಪು ಕಡಿಮೆ ಆಗುತ್ತದೆ. ಹಾಗಾಗಿ w11 ಸ್ವಲ್ಪ (2.0 → 2.0027) ಹೆಚ್ಚಿತು. ಬದಲಾವಣೆ ಚಿಕ್ಕದು ಏಕೆಂದರೆ $\sigma(1-\sigma) = 0.083$ ಚಿಕ್ಕದು (ಔಟ್‌ಪುಟ್ 0.909 ಈಗಾಗಲೇ 1 ಕ್ಕೆ ಹತ್ತಿರ, ಸಮತಟ್ಟಾದ ಭಾಗ). ಸ್ಲೈಡ್‌ನಲ್ಲಿ ಸುತ್ತಿದ ಸಂಖ್ಯೆಗಳಿಂದ 2.002656 ಎಂದು ಬರೆದಿದೆ; ನಿಖರ ಲೆಕ್ಕದಲ್ಲಿ ≈ 2.00265 — ಇದೇ ಉತ್ತರ.

ನೀವೇ ಪ್ರಯತ್ನಿಸಿ (ಸ್ಲೈಡ್‌ನ “On your own!” ಕಾರ್ಯ)

w22 ಅನ್ನು ಬದಲಿಸಿ: w22 = 4, e2 = 0.5, α = 0.1. (ಚಿತ್ರದಲ್ಲಿ w12 = 1.0, $o_1 = 0.4$, $o_2 = 0.5$ ಎಂದು ಭಾವಿಸಿ.)

ಉತ್ತರ: x = (1.0×0.4) + (4.0×0.5) = 2.4; σ(2.4) = 0.917; σ(1−σ) = 0.0763; dE/dw22 = −0.5 × 0.0763 × 0.5 = −0.0191; ಹೊಸ w22 = 4 + 0.1×0.0191 = **4.0019**.

೧೪. ಸಾರಾಂಶ, ಪದಕೋಶ ಮತ್ತು ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

ಪೂರ್ಣ ಕಲಿಕೆಯ ಚಕ್ರ — ಒಂದೇ ನೋಟದಲ್ಲಿ

ಹಂತ	ಕೆಲಸ	ಬಳಸುವ ಸೂತ್ರ
೧. Forward	ಪ್ರತಿ ಪದರದಲ್ಲಿ ಮೊತ್ತ ಮತ್ತು activation	$X = W \cdot I$, $O = \sigma(X)$
೨. Error	ಸರಿಯಾದ ಉತ್ತರದೊಂದಿಗೆ ಹೋಲಿಕೆ	$e = t - o$, $E = (t - o)^2$
೩. Backprop	ತಪ್ಪನ್ನು ಹಿಂದಕ್ಕೆ ಹಂಚು	$\text{error_hidden} = W^T \cdot \text{error_output}$
೪. Gradient	ಪ್ರತಿ ತೂಕದ ಹೊಣೆ ಲೆಕ್ಕ	$dE/dw = -e \cdot \sigma(1-\sigma) \cdot o_j$
೫. Update	ತೂಕ ಸ್ವಲ್ಪ ಬದಲಿಸು	$w_{\text{ಹೊಸ}} = w - \alpha \cdot dE/dw$
೬. Repeat	ತಪ್ಪು ಸಾಕಷ್ಟು ಚಿಕ್ಕದಾಗುವವರೆಗೆ	ಅನೇಕ ಸುತ್ತುಗಳು (epochs)

ಪದಕೋಶ

ಇಂಗ್ಲಿಷ್	ಕನ್ನಡದಲ್ಲಿ ಸರಳ ಅರ್ಥ
Perceptron	ಒಂದೇ ಕೃತಕ ನರಕೋಶ: ತೂಕ ಮೊತ್ತ + ಹೌದು/ಇಲ್ಲ ಗೇಟ್
Weight (ತೂಕ)	ಒಂದು ಇನ್‌ಪುಟ್ ಎಷ್ಟು ಮುಖ್ಯ ಎಂದು ಹೇಳುವ ಸಂಖ್ಯೆ — ಇದನ್ನೇ ಕಲಿಯುವುದು
Bias (ಬಯಾಸ್)	ನಿರ್ಧಾರ ರೇಖೆಯನ್ನು ಸರಿಸುವ ಸ್ಥಿರ ಸಂಖ್ಯೆ
Activation function	ಮೊತ್ತವನ್ನು ಔಟ್‌ಪುಟ್ ಆಗಿ ಬದಲಿಸುವ ಗೇಟ್ (step, sigmoid…)
Linearly separable	ಒಂದು ನೇರ ರೇಖೆಯಿಂದ ಬೇರ್ಪಡಿಸಬಹುದಾದ ದತ್ತಾಂಶ (AND ಹೌದು, XOR ಇಲ್ಲ)
Hidden layer (ಮರೆ ಪದರ)	ಇನ್‌ಪುಟ್ ಮತ್ತು ಔಟ್‌ಪುಟ್ ನಡುವಿನ ಪದರ — ಹೆಚ್ಚು ಸಂಕೀರ್ಣ ಮಾದರಿಗಳ ಕಲಿಕೆಗೆ
Feedforward	ಸಂಕೇತ ಒಂದೇ ದಿಕ್ಕಿಗೆ (ಇನ್‌ಪುಟ್ → ಔಟ್‌ಪುಟ್) ಹರಿಯುವ ನೆಟ್‌ವರ್ಕ್
Loss / Error	ನೆಟ್‌ವರ್ಕ್ ಎಷ್ಟು ತಪ್ಪು ಎಂದು ಹೇಳುವ ಸಂಖ್ಯೆ (MAE, MSE)
Backpropagation	ತಪ್ಪನ್ನು ಹಿಂದಕ್ಕೆ ಹಂಚಿ, ಪ್ರತಿ ತೂಕದ ಹೊಣೆ ಕಂಡುಹಿಡಿಯುವ ವಿಧಾನ
Gradient (ಗ್ರೇಡಿಯಂಟ್)	ತಪ್ಪು ಅತಿ ವೇಗವಾಗಿ ಹೆಚ್ಚುವ ದಿಕ್ಕು; ಇಳಿಯಲು ಅದರ ವಿರುದ್ಧ ಹೋಗುತ್ತೇವೆ
Learning rate	ಒಂದು ಬಾರಿಗೆ ತೂಕ ಎಷ್ಟು ಬದಲಾಗಬೇಕು — ಹೆಜ್ಜೆಯ ಗಾತ್ರ
Epoch	ಎಲ್ಲ ತರಬೇತಿ ಉದಾಹರಣೆಗಳನ್ನು ಒಮ್ಮೆ ತೋರಿಸಿದ ಒಂದು ಸುತ್ತು

ಅಭ್ಯಾಸ ಪ್ರಶ್ನೆಗಳು

1. ಕಾರು ಉದಾಹರಣೆಯಲ್ಲಿ (x1, x2, x3) = (0, 1, 1) ಆದರೆ ತೀರ್ಮಾನ ಏನು? (ತೂಕ 0.6, 0.4, −0.5; ಮಿತಿ 0.5)

2. w1 = 2, w2 = 1, b = −4 ಇರುವ ಪರ್ಸೆಪ್ಟ್ರಾನ್‌ಗೆ (x1, x2) = (2, 1) ನೀಡಿದರೆ ಔಟ್‌ಪುಟ್ ಎಷ್ಟು? (step: > 0 ಆದರೆ 1)
3. ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಕಲಿಕೆ ನಿಯಮದಲ್ಲಿ t = 1, o = 0, η = 0.1, x = 1 ಆದರೆ ತೂಕ ಎಷ್ಟು ಬದಲಾಗುತ್ತದೆ?
4. XOR ಅನ್ನು ಒಂದೇ ಪರ್ಸೆಪ್ಟ್ರಾನ್ ಕಲಿಯಲಾಗದಿರಲು ಕಾರಣ ಏನು?
5. σ(0) ಎಷ್ಟು? σ ಯ ವ್ಯುತ್ಪನ್ನದ ಗರಿಷ್ಠ ಮೌಲ್ಯ ಎಷ್ಟು?
6. ಗ್ರೇಡಿಯಂಟ್ ಡಿಸೆಂಟ್‌ನಲ್ಲಿ $E(w) = (w - 3)^2$, w = 0, η = 0.1 ಆದರೆ ಮೊದಲ ಹೆಜ್ಜೆಯ ನಂತರ w ಎಷ್ಟು?
7. η ತುಂಬ ದೊಡ್ಡದಿದ್ದರೆ ಏನಾಗುತ್ತದೆ? ತುಂಬ ಚಿಕ್ಕದಿದ್ದರೆ?

ಉತ್ತರಗಳು: (೧) Σ = 0.4 − 0.5 = −0.1 < 0.5 → **ಬೇಡ (0)**. (೨) 2×2 + 1×1 − 4 = 1 > 0 → **1**. (೩) Δw = 0.1 × (1 − 0) × 1 = **+0.1**. (೪) ಹಸಿರು ಮತ್ತು ಕೆಂಪು ಬಿಂದುಗಳನ್ನು ಒಂದು ನೇರ ರೇಖೆಯಿಂದ ಬೇರ್ಪಡಿಸಲಾಗದು; ಮರೆ ಪದರ ಬೇಕು. (೫) σ(0) = **0.5**; ಗರಿಷ್ಠ ವ್ಯುತ್ಪನ್ನ = **0.25**. (೬) ಗ್ರೇಡಿಯಂಟ್ = 2(0−3) = −6; w = 0 − 0.1×(−6) = **0.6**. (೭) ದೊಡ್ಡದು: ಕನಿಷ್ಠವನ್ನು ದಾಟಿ ಹಾರುತ್ತದೆ, ನೆಲೆಸುವುದಿಲ್ಲ; ಚಿಕ್ಕದು: ಕಲಿಕೆ ಬಹಳ ನಿಧಾನ.